



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

POINTER

ผศ.ดร.เจียบวุฒิ รัตนวิไลสกุล chaibwoot.r@sci.kmutnb.ac.th

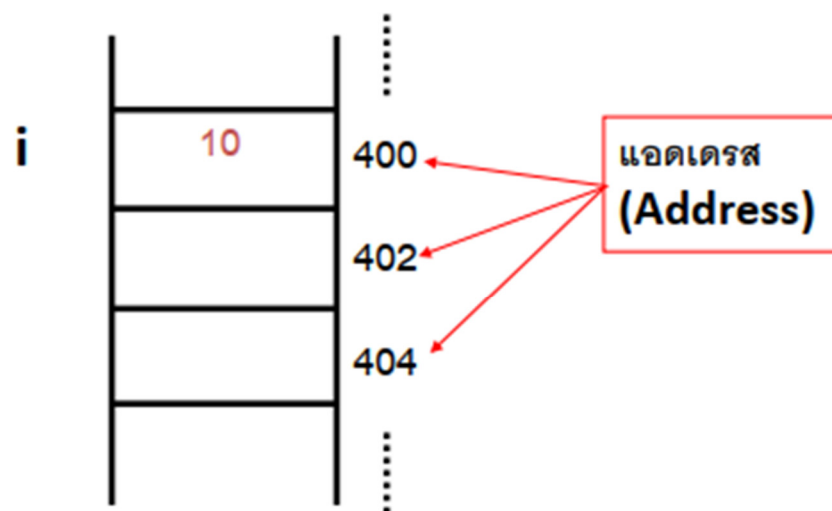
ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

พอยน์เตอร์

- เป็นชนิดข้อมูลชนิดหนึ่งของภาษา C (int, float, char)
- **มีความเร็วในการทำงานสูง**
- ช่วยประหยัดเนื้อที่ในหน่วยความจำหลักขณะประมวลผล เมื่อเทียบกับอาร์เรย์ (array)
- ใช้ตัวชี้ร่วมกับฟังก์ชันเพื่อเพิ่มประสิทธิภาพการเขียนโปรแกรม
- ตัวแปรชนิดพอยน์เตอร์จะเก็บค่าที่อยู่ของหน่วยความจำหลัก ซึ่งต่างกับตัวแปรปกติที่เก็บค่าที่แท้จริงของข้อมูล
- การใช้ตัวแปรชนิดพอยน์เตอร์จะเป็นการเข้าถึงข้อมูลหรือเป็นการอ้าง ถึงตำแหน่งที่เก็บข้อมูล
- การประกาศตัวแปรประเภทพอยน์เตอร์
`int *ip;` เป็นการประกาศตัวแปร ip ให้เป็นตัวแปรพอยน์เตอร์ที่ชี้ไปยังตัวแปรประเภท int
`double *dp;` เป็นตัวแปรพอยน์เตอร์ที่ชี้ไปยังตัวแปรประเภท double

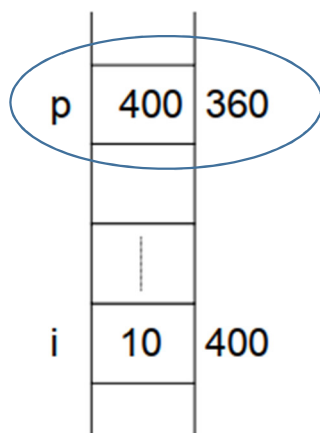
ภาพจำลองการแทนข้อมูลในหน่วยความจำแบบปกติ

```
int i;  
i = 10;
```



```
int i = 10;  
int *p = &i;
```

```
cout<<i<<endl;  
cout<<p<<endl;  
cout<<&i<<endl;  
cout<<*p<<endl;
```



ตัวแปร pointer

`int *p = &i;` คือ `p` เก็บค่าของตำแหน่ง คือ `400`
และในช่อง `400` หรือ `i` เก็บ ค่าจริงๆ คือ `10`

การประกาศตัวแปรประเภทตัวชี้

```
int x = 1, y = 2;
```

```
int *ip, *iq;
```

* คือ ค่า

ตัวมัน คือ address

```
ip = &x;
```

```
y = *ip;
```

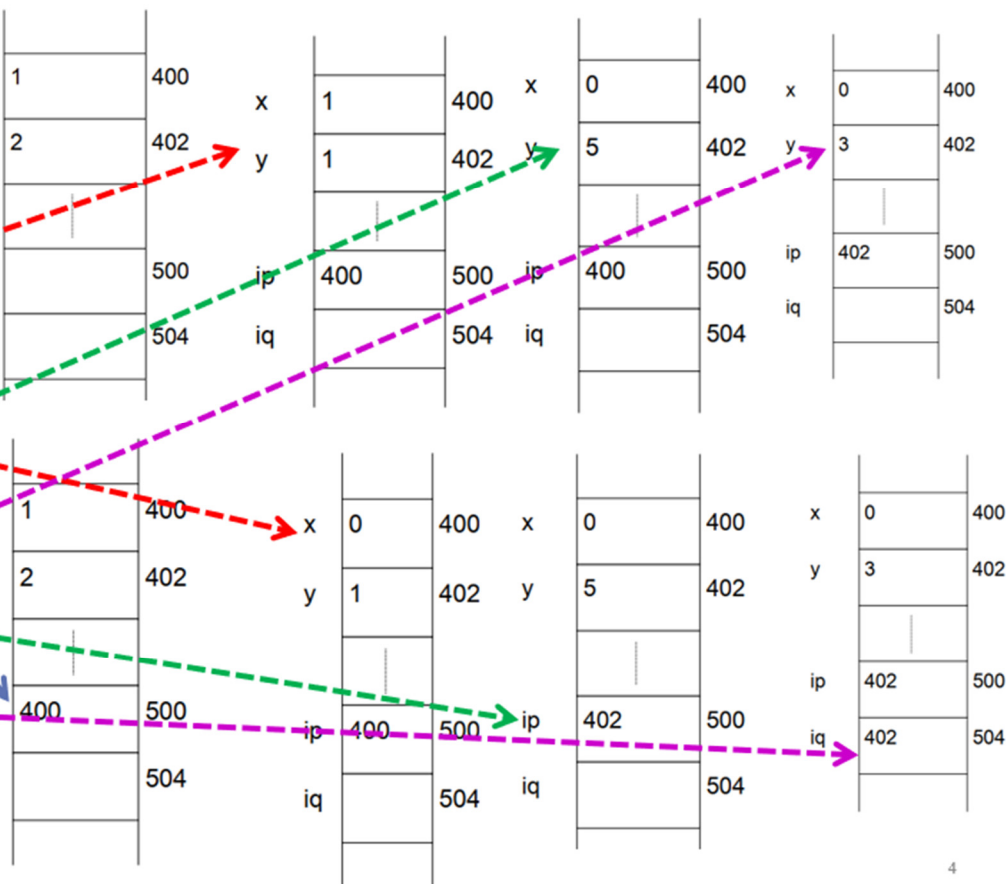
```
*ip = 0;
```

```
y = 5;
```

```
ip = &y;
```

```
*ip = 3;
```

```
iq = ip;
```



ตัวชี้และอาร์กิวเมนต์ของฟังก์ชัน

- เนื่องจากภาษาซีมีการส่งอาร์กิวเมนต์ให้กับฟังก์ชันแบบ By Value และฟังก์ชันสามารถคืนค่า (return) **ค่าได้เพียงหนึ่งค่า**หากต้องการให้ฟังก์ชันมีการเปลี่ยนแปลงค่าและคืนค่ากลับมา ยัง **ฟังก์ชันที่เรียกใช้มากกว่าหนึ่งค่า**จะต้องนำพอยน์เตอร์เข้ามาช่วย
- อาร์กิวเมนต์ที่เป็นประเภทพอยน์เตอร์จะช่วยให้ **ฟังก์ชันสามารถเปลี่ยนค่าให้กับตัวแปร**ที่ส่งเข้ามาได้ เนื่องจากอาร์กิวเมนต์นั้นจะเก็บแอดเดรสของตัวแปรที่ส่งเข้ามา เมื่อมีการเปลี่ยนแปลงค่าของ **อาร์กิวเมนต์ผ่าน reference** ค่าของตัวแปรที่ส่งเข้ามาจะถูกเปลี่ยนค่าพร้อมกันในทันที
- จากตัวอย่างนี้ เราไม่ต้อง return แต่เราสามารถกำหนดค่าให้ตัวแปรได้เลย คือตัวแปร x กับ y โดยอาศัยการแก้ค่าโดย pass by reference

```
void swap (int *px, int *py)
{
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}

int main ( )
{
    int x = 5, y = 10;
    printf("Before swap : x = %d", x, ",y = %d\n", y);
    swap ( &x, &y);
    printf("After swap : x = %d", x, ", y = %d\n", y);
}
```

ตัวชี้กับอาร์เรย์

- อาร์เรย์เป็นประเภทข้อมูลที่เก็บชุดของข้อมูลประเภทเดียวกัน มักใช้กับการทำงานที่ต้องทำงานกับตัวแปรชนิดเดียวกันหลายตัวที่มี การทำงานเหมือนกัน เช่น คะแนนของนักศึกษาภายในห้อง 20 คน เป็นต้น อาร์เรย์ในภาษาซีจะนำหลักการของพอยน์เตอร์เข้ามาใช้
- การทำงานใด ๆ ของอาร์เรย์สามารถใช้พอยน์เตอร์เข้ามาแทนที่
- อาร์เรย์ = pointer

```
int v[4]; char s1[2]; float f[10];
```

```
v[0] = 10; Start at 0
```

```
v[3] = 15; END
```

```
v[4] = 5; Error ค่าที่ผิดไป การอ้างข้อมูลต้องอยู่ในขอบเขต
```



ให้อ่านค่าของจำนวนเต็ม 5 จำนวนจากคีย์บอร์ด และแสดงผลในลำดับที่กลับกัน

```
# include <stdio.h>
# define SIZE 5
main ( )
{
    int table[SIZE];
    for (int k = 0; k < SIZE; k++)
        scanf ("%d", &table[k]);
    for (int k = SIZE-1; k >= 0; k--)
        printf ("%d\n", table[k]);
}
```



การใช้ตัวชี้กับอาร์เรย์

```
int a[10];
```

```
int *pa;
```

กำหนดให้พอยน์เตอร์ pa ชี้ไปยังอาร์เรย์ a ด้วยคำสั่ง

```
pa = &a[0];
```

หรือ

```
pa = a;
```

pa จะเก็บค่าแอดเดรสเริ่มต้นของอาร์เรย์ a

```
pa+1;
```

การอ้างอิงแอดเดรสของ a[1]

```
*(pa+1);
```

ค่าของ a[1]

```
char *s;
```

```
char s[];
```

f (&a[2]) หรือ f (a+2) เป็นการส่ง แอดเดรสของสมาชิก a[2]
ให้กับฟังก์ชัน f สามารถทำได้โดยการประกาศ

```
f (int arr[]) { ..... }
```

```
f (int *arr) { ..... }
```

ตัวชี้ตัวอักษรและฟังก์ชัน

```
int strlen (char *s)
{
    char *p = s;
    while (*p != '\0')
        p++;
    return p-s;
}
```

เนื่องจาก s ชี้อยู่ที่ตำแหน่งเริ่มต้น โดยมี p ชี้ไปที่ s เช่นเดียวกัน แต่จะมีการเลื่อน p ไปทีละหนึ่งตำแหน่ง จนกว่าค่าที่ตำแหน่งที่ p ชี้อยู่จะเท่ากับ '\0' เมื่อนำ p ค่าสุดท้ายมาลบกับ s ที่ตำแหน่ง เริ่มต้นก็จะได้ ความยาวของข้อมูลที่ส่งเข้ามา



- message
- | | | | | | | | | | | | | |
|--|---|---|---|---|---|---|---|---|---|---|---|---|
| | H | e | l | l | o | , | w | o | r | l | d | \ |
|--|---|---|---|---|---|---|---|---|---|---|---|---|
- amessage
- | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| H | e | l | l | o | , | w | o | r | l | d | \ |
|---|---|---|---|---|---|---|---|---|---|---|---|

ตัวชี้ตัวอักษรและฟังก์ชัน

strcpy () ➔ Array

```
void strcpy ( char *s, char *t )  
{  
    int i=0;  
    while( ( s[i] = t[i] ) != '\0' )  
        i++;  
}
```

strcpy () ➔ Point

```
void strcpy ( char *s, char *t )  
{  
    while ( ( *s = *t ) != '\0' )  
    {  
        s++;  
        t++;  
    }  
}
```



การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

- กรณีการส่งอาร์กิวเมนต์เป็นตัวแปร struct จะไม่เหมาะกับ struct ที่มีขนาดใหญ่ เนื่องจากทุกครั้งที่ส่งตัวแปร struct จะเป็นการสำเนาตัวแปรตัวใหม่ขึ้นมาในฟังก์ชันซึ่งจะทำให้ช้าและเปลืองพื้นที่หน่วยความจำ เราจะใช้พอยน์เตอร์เข้ามาช่วยแก้ปัญหานี้
- โดยส่งแอดเดรสของตัวแปร struct มายังฟังก์ชันซึ่งรับอาร์กิวเมนต์ เป็นพอยน์เตอร์อาร์กิวเมนต์จะชี้ไปยังแอดเดรสเริ่มต้นของตัวแปร struct จะช่วยให้การทำงานเร็วขึ้นและเปลืองหน่วยความจำน้อยลง แต่สิ่งที่ต้องระวังคือ หากมีการเปลี่ยนแปลงค่าที่อาร์กิวเมนต์พอยน์เตอร์ชี้อยู่ ค่าในตัวแปร struct ที่ส่งมายังฟังก์ชันจะเปลี่ยนตามโดยอัตโนมัติ



การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

```
struct point origin;
```

```
point *pp;
```

```
pp = &origin;
```

```
printf ( "origin is (%d, %d)\n", (*pp).x, (*pp).y );
```

```
printf ( "origin is (%d, %d)\n", pp->x, pp->y);
```

(*pp).x จะไม่เหมือนกับ *pp.x เนื่องจากเครื่องหมาย . จะมีลำดับความสำคัญสูงกว่า *

- การอ้างถึงสมาชิกโครงสร้างโดยใช้เครื่องหมาย ->

```
ptrdate->day = 7;
```

```
scanf ( "%d", &ptrdate->year );
```

- การอ้างถึงสมาชิกของโครงสร้างผ่านตัวแปรพอยน์เตอร์

```
(*ptrdate).day = 7;
```

```
scanf ( "%d", &((*ptrdate).year) );
```

การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

Date today;

Date *ptrdate;
```

แบบที่ 1

การประกาศแบบ
ข้อมูลโครงสร้าง

การประกาศตัวแปร
ข้อมูลแบบโครงสร้าง

การประกาศตัวแปร
pointer ชี้ไปยังโครงสร้าง

```
struct date {
    int day;
    int month;
    int year;
} *ptrdate;
```

แบบที่ 2

```
typedef struct {
    int day;
    int month;
    int year;
} Date;

typedef Date *PtrDate;

PtrDate ptrdate;
```

แบบที่ 3

การประกาศแบบ
ข้อมูลโครงสร้าง

การประกาศประเภท
ตัวแปร pointer ชี้ไปยัง
โครงสร้าง

การประกาศตัวแปร
pointer ชี้ไปยัง โครงสร้าง



การประกาศตัวแปรชี้ (pointer) ชี้ไปยัง STRUCT

```
struct date
{
    int day;
    int month;
    int year;
};
main ( )
{
    struct date    today;
    struct date    *ptrdate;

    ptrdate = &today;
    ptrdate->day = 27;
    ptrdate->month = 9;
    ptrdate->year = 1985;
    printf ( "Today\'s date is %2d/%2d/%4d\n", ptrdate->day, ptrdate->month, ptrdate->year );
}
```



การใช้งาน Pointer

```
#include<stdio.h>
```

```
void main( )
```

```
{
```

```
    int N = 100;
```

```
    int *pN;
```

ประกาศ

```
    pN = &N;
```

เชื่อม

```
    printf ( "%p\n", &N );
```

```
    printf ( "%p\n", pN );
```

ตัวมันเป็น address

```
    printf ( "%d\n", N );
```

```
    printf ( "%d\n", *pN );
```

ค่าของมัน

```
}
```



การใช้งาน Pointer of FUNCTION

```
#include<stdio.h>
void sum_value( int a, int b) //pass by value
{
    a = 3;
    b = 4;
    printf( "%d %d\n", a, b );
}
```

```
void sum_ref( int *a, int *b) //pass by reference
{
    *a = 5;
    *b = 6;
    printf( "%d %d\n", *a, *b );
}
void main( )
{
```

```
    int x = 1;
    int y = 2;
    printf( "%d %d\n",x, y );
    sum_value(x,y);
    printf( "%d %d\n",x, y );
    sum_ref(&x,&y);
    printf( "%d %d\n",x, y );
}
```

1 กับ 2
3 กับ 4
เหมือนเดิม 1 กับ 2
ส่ง address ให้ x กับ y คือ 5 กับ 6
x กับ y คือ 5 กับ 6



การใช้งาน Pointer of Array

```
#include<stdio.h>
```

```
void main( )
```

```
{
```

```
    int x[] = {1,2,3,4,5};
```

```
    printf( "%p\n", x);
```

```
    int *y = x;
```

```
    printf( "%p\n", y);
```

```
    printf( "%d\n", x[0]);
```

```
    printf( "%d\n", *(y+0) );
```

```
    printf( "%d\n", x[1]);
```

```
    printf( "%d\n", *(y+1) );
```

```
}
```

x คือ pointer และ x คือ Array

อ้างอิงตำแหน่ง

อ้างอิงตำแหน่ง

อ้างอิงตำแหน่ง

อ้างอิงตำแหน่ง



การใช้งาน Pointer of String

```
char *s1 = "my name is be1";
printf( "%s\n", s1);
printf( "%c\n", *(s1+1) );
char s2[] = "my name is be2";
printf( "%s\n", s2);
printf( "%c\n", s2[3] );
s1 = "I love U";
//s2 = "I love U";
//*(s1+2) = 'x';
s2[2] = 'x';
printf( "%s\n", s2);
char *s3[] = {"s1","s2","s3"};
printf( "%s\n", s3[0] );
s3[1] = "GOD";
printf( "%s\n", s3[1] );
char *s4 = s2;
s4[2] = 'k';
printf( "%s\n", s2);
printf( "%s\n", s4);
```

ข้อความเป็น pointer

แสดงทั้งหมด

เอาค่าตัวเดียว

ข้อความเป็น []

เอาค่าตัวเดียว

เปลี่ยนทั้งอันทำได้

//Error ถ้าเป็น Array ทำไม่ได้

//Error

ทำได้

ข้อความเป็น pointer + array

การ copy string ต้องระวัง เพราะถ้าเขียน char *s4 = s1;

ถ้าต้นฉบับเป็น Array สามารถแก้ไขค่าได้

s1 กับ s4 เชื่อมกัน ถ้าเปลี่ยน เปลี่ยนทั้งคู่



การใช้งาน Pointer of String

```
char *s1 = "test";  
char s2[4];  
char s3[4];  
char *s4 = s3;
```

```
printf( "%s\n", s1);
```

สามารถทำได้

```
scanf("%s",s1);  
printf( "%s\n", s1);
```

ไม่สามารถทำได้ ถ้าเป็น **pointer**
สามารถทำได้

```
scanf("%s",s2);  
printf( "%s\n", s2);
```

สามารถทำได้ เป็น **Array** สามารถทำได้
สามารถทำได้

```
scanf("%s",s4);  
printf( "%s\n", s4);
```

สามารถทำได้ ถ้ากรณีเป็น **Array** มาก่อน
สามารถทำได้



การใช้งาน Pointer of String

```
char name[100][100];  
scanf("%s",name[i]);
```

กรณีใช้ String Array ควรใช้ Array 2 มิติ
อ้างอิง Array 2 มิติ

```
int num[100];  
scanf("%d",&num[i]);
```

หรืออาจจะใช้ pointer ก็ได้

ตัวอย่างการ Copy String และ Compare String

```
#include<string.h>
```

```
char str1[20] = "C programming";  
char str2[20];  
strcpy(str2, str1);
```

```
char str1[20] = "C programming";  
char str2[20] = "C programming";  
if( strcmp( id[i], pass ) == 0 )  
{  
}
```



การใช้งาน Pointer of STRUCT

```
#include<stdio.h>
```

```
#include<string.h>
```

```
struct Books
```

```
{
```

```
    char title[50];
```

```
    int book_id;
```

```
};
```

```
void printBook( struct Books *book )
```

```
{
```

```
    printf( "Book title : %s\n", book->title);
```

```
    printf( "Book book_id : %d\n", book->book_id);
```

```
}
```

```
void main( )
```

```
{
```

```
    struct Books Book1;
```

```
    strcpy( Book1.title, "C Programming");
```

```
    Book1.book_id = 6495407;
```

```
    printf( "Book 1 title : %s\n", Book1.title);
```

```
    printf( "Book 1 book_id : %d\n", Book1.book_id);
```

```
    printBook( &Book1 );
```

```
}
```

ควร import string เพราะต้องใช้สำหรับการใส่ค่า

struct ใส่ function ข้างในไม่ได้ จึงต้อง pass parameter โดย pointer

รับค่าเป็น pointer

ถ้ากรณีอ้างอิงผ่าน pointer ใช้ ->

สร้าง structure

พยายามใช้ฟังก์ชันสำเร็จรูป

ถ้ากรณีปกติใช้ "."



ค่ายโอลิมปิกวิชาการ วิชาคอมพิวเตอร์

Linked List

ผศ.ดร.เจียบวุฒิ รัตนวิไลสกุล chaibwoot.r@sci.kmutnb.ac.th

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

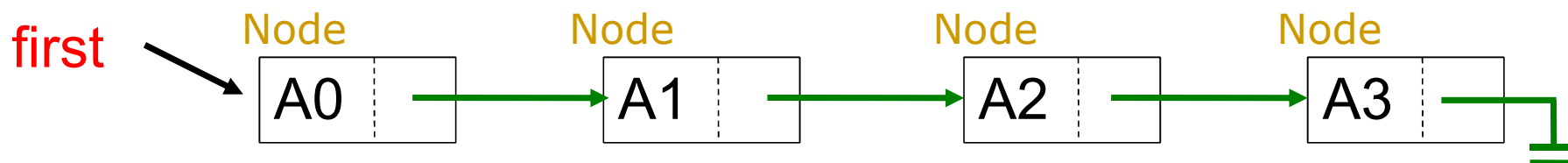


Outline

- Linked list nodes
- Linked list operations
 - Insertion
 - Append
 - Deletion
- Linked list representation & implementation
- Other types of linked lists
 - Sorted
 - Doubly-linked
 - Circular

Linked Lists

- รายการโยง ประกอบด้วยกลุ่มของโหนดซึ่งแต่ละโหนดประกอบด้วยเขตข้อมูลอย่างน้อยสองเขต(field)
 - ข้อมูลในโหนด (data)
 - ตัวชี้หรือรายการโยง บอกถึงที่อยู่ของโหนดถัดไปในรายการ (reference to the next Node in the list)
- การเพิ่ม(addition) หรือ การลบ(deletion) ข้อมูลที่อยู่ในรายการใช้เวลา $O(n)$.

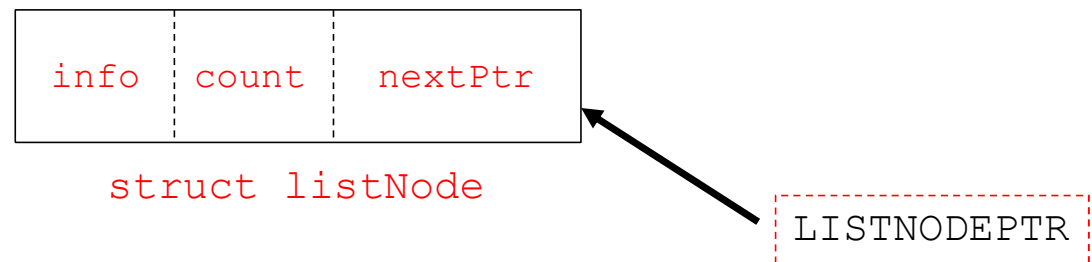




Struct Linked Lists

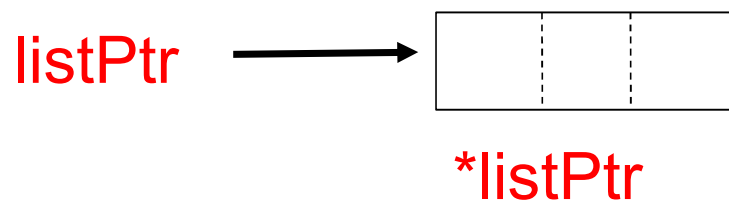
```
typedef struct listNode
{
    char info;
    int count;
    struct listNode *nextPtr;
} my_listNode;
```

```
main()
{
    my_listNode LISTNODE;
    my_listNode *LISTNODEPTR;
}
```



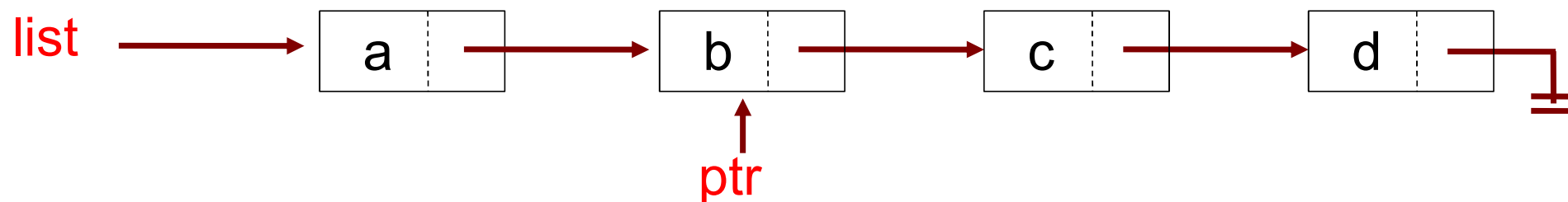
การนำตัวชี้มาใช้งาน

- การเขียนสัญลักษณ์ * ตามด้วยชื่อตัวชี้ จะแทนโหนดของข้อมูลซึ่งตัวชี้ชี้อยู่ เช่น จากการประกาศชนิดตัวชี้ของ listNode การเขียน nextPtr อ้างถึงที่อยู่ของโหนดหนึ่งในลิงค์ลิสต์ ที่ตัวชี้ nextPtr ชี้ไปถึง
- listPtr แตกต่างจาก *listPtr เพราะ listPtr คือตัวชี้ที่อ้างถึงเลขที่อยู่ แต่ *listPtr คือข้อมูลที่ถูกระบุ
- สามารถชี้ไปที่เขตต่าง ๆ ของฟิลด์ได้ โดย ชื่อตัวชี้->ชื่อฟิลด์



การจัดการกับโหนดชนิดตัวชี้

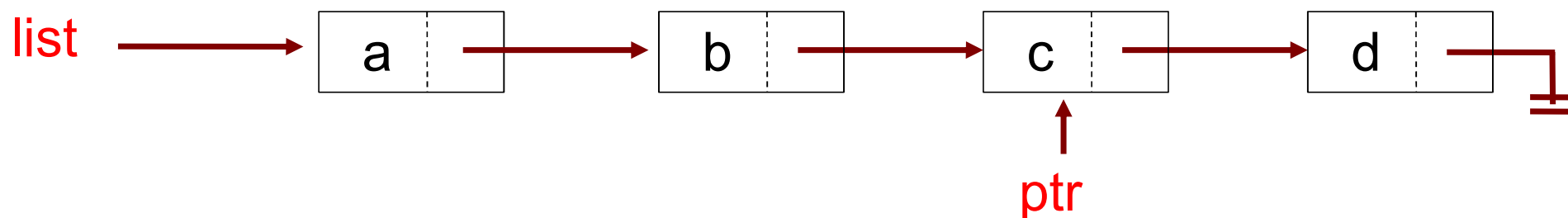
- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode



- list และ ptr เป็นตัวชี้โดย list ชี้ไปที่โหนดแรก และ ptr ชี้ไปที่โหนดที่สอง

การจัดการกับโหนดชนิดตัวชี้

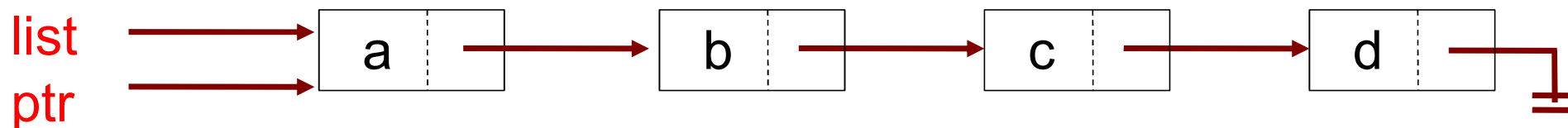
- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode



- การสั่งให้ ptr ชี้ไปที่โหนดถัดไป โดยใช้คำสั่ง `ptr = ptr->nextPtr;`

การจัดการกับโหนดชนิดตัวชี้

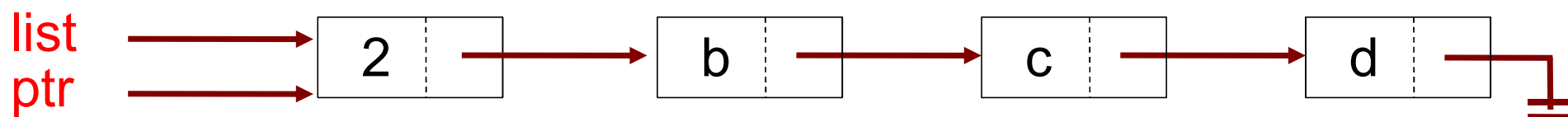
- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode



- การสั่งให้ ptr ชี้ไปที่โหนดแรกของรายการ โดย `ptr = list;`

การจัดการกับโหนดชนิดตัวชี้

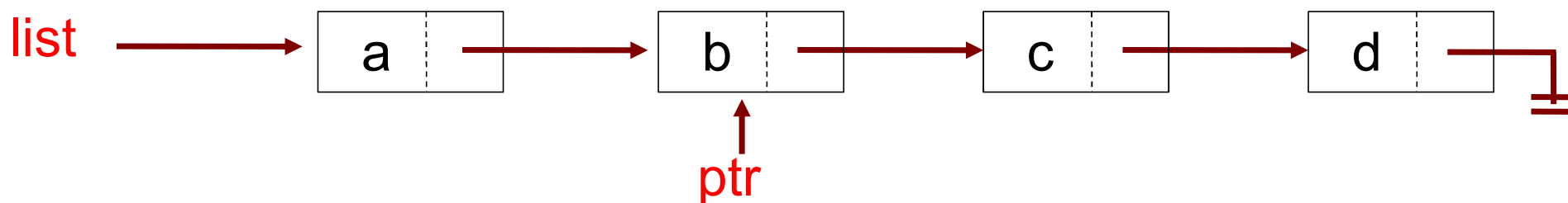
- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode



- ให้ค่าของเขต **info** ในโหนดแรกของลิงค์ลิสต์มีค่าเท่ากับ 2 `list->info = 2;`

การจัดการกับโหนดชนิดตัวชี้

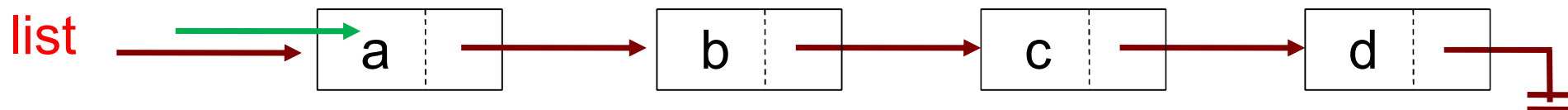
- กำหนดให้ ptr และ ptr->nextPtr เป็นตัวชี้ (Pointer) ชี้ไปที่โครงสร้างข้อมูลที่เป็น listNode



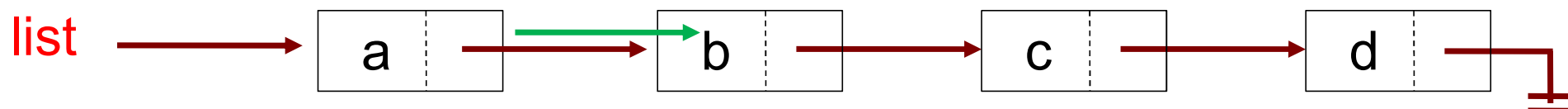
- ให้ ptr ชี้ไปที่โหนดที่สองในลิงค์ลิสต์ โดย ptr=list->nextPtr

การเรียกใช้ตัวชี้ที่ตำแหน่งต่าง ๆ ในลิงค์ลิสต์

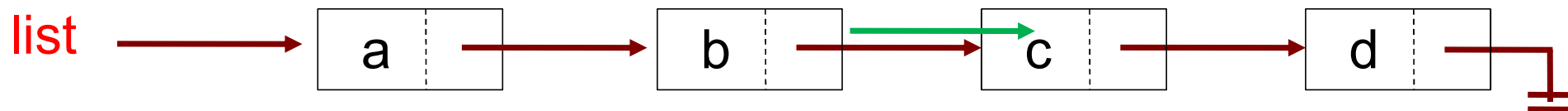
- list ชี้ไปที่โหนดแรกของรายการ



- list->next ชี้ไปที่โหนดที่ 2 ของรายการ



- list->next->next ชี้ไปที่โหนดที่ 3 ของรายการ



การทำงานของลิงค์ลิสต์

- ในการค้นหา อ่าน และการพิมพ์ข้อมูลจากลิงค์ลิสต์สามารถทำได้โดยการท่องเที่ยวในลิงค์ลิสต์ โดยเริ่มต้นจากกำหนด ptr ที่จุดเริ่มต้นของรายการด้วยคำสั่ง ptr=list
- ต่อมาสามารถเลื่อนตัวชี้ไปที่โหนดถัดไปโดย ptr=ptr->next

```
typedef struct node
{
    int info;
    int count;
    struct node *next;
} my_node;
```

```
main()
{
    my_node *ptrNode;
}
```

ชนิดข้อมูล struct node



ชนิดข้อมูล listPtr

list

การสร้างลิงค์ลิสต์

- การสร้างลิงค์ลิสต์

```
void createList()  
{  
    my_node *list = malloc(sizeof(my_node));  
    if(list)  
    {  
        list -> info = 0;  
        list -> count = 0;  
        list -> next = NULL;  
    }  
}
```

- การทดสอบว่าลิงค์ลิสต์ว่างหรือไม่

```
int isEmpty(my_node list)  
{  
    return (list->count==0);  
}
```

การสร้างและการทำลายโหนด

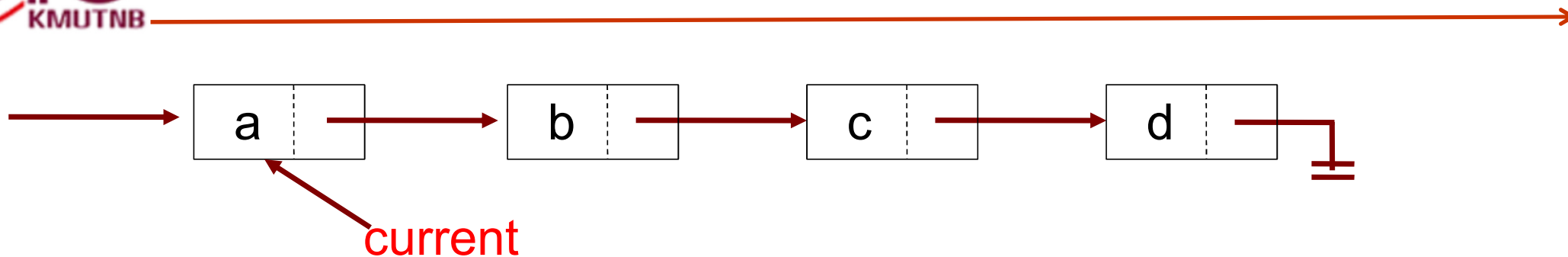
- การทำงานเกี่ยวกับแถวลำดับหรือระเบียบ เมื่อผู้ใช้ประกาศชื่อโหนดในโปรแกรมแล้วนำโปรแกรมไปแปลชุดคำสั่ง เครื่องคอมพิวเตอร์จะกำหนดเนื้อที่ในหน่วยความจำให้โหนด แต่สำหรับโหนดตัวชี้ซึ่งทำงานแบบพลวัต ภาษาซีมีฟังก์ชัน `malloc` เพื่อจัดเนื้อที่ในหน่วยความจำสำหรับโหนดชนิดนี้
- ฟังก์ชัน `malloc` ต้องการพารามิเตอร์ 1 ตัวที่เป็นจำนวนไบต์ที่ต้องการจองเนื้อที่
- รูปแบบ
 - `malloc(จำนวนไบต์ที่ต้องการจอง)`
 - เช่น `malloc(sizeof(x))` เมื่อ `x` เป็นชื่อโหนด



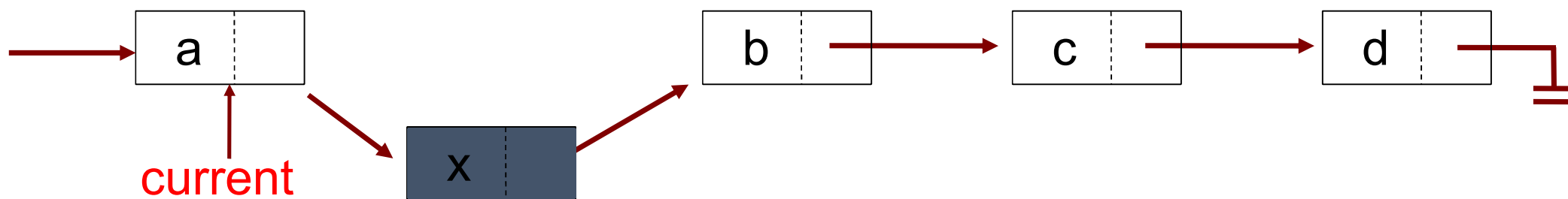
การสร้างและการทำลายโหนด

- เมื่อผู้ใช้ไม่ต้องการใช้ตัวแปรนั้นอีกต่อไปสามารถยกเลิกเนื้อที่สำหรับตัวแปรนั้นเพื่อให้ส่วนอื่นนำไปใช้ได้
- ฟังก์ชันที่ใช้คือ free
- รูปแบบ `free(ptr);`
 - ptr คือ โหนดชนิดตัวชี้ซึ่งชี้ไปที่หน่วยความจำที่ต้องการยกเลิกการใช้งาน

การเพิ่มโหนดในลิงค์ลิสต์



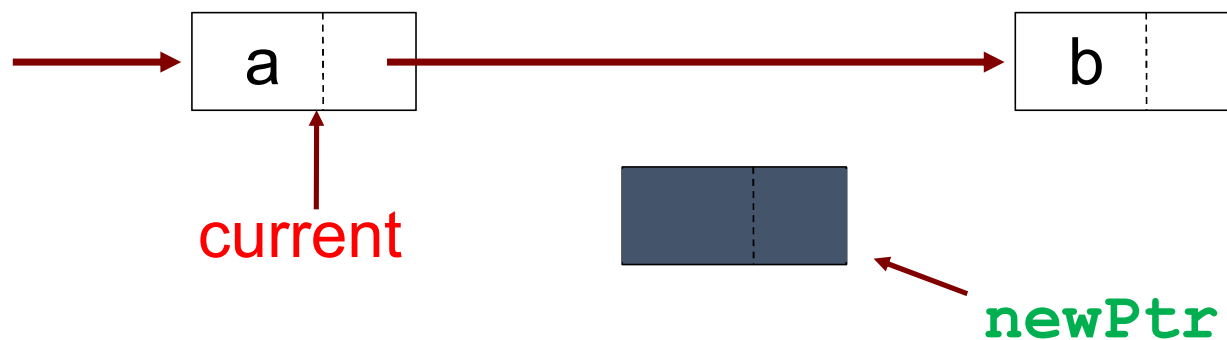
- เพิ่มโหนด X หลังตำแหน่งที่ current ชี้



การเพิ่มโหนดในลิงค์ลิสต์

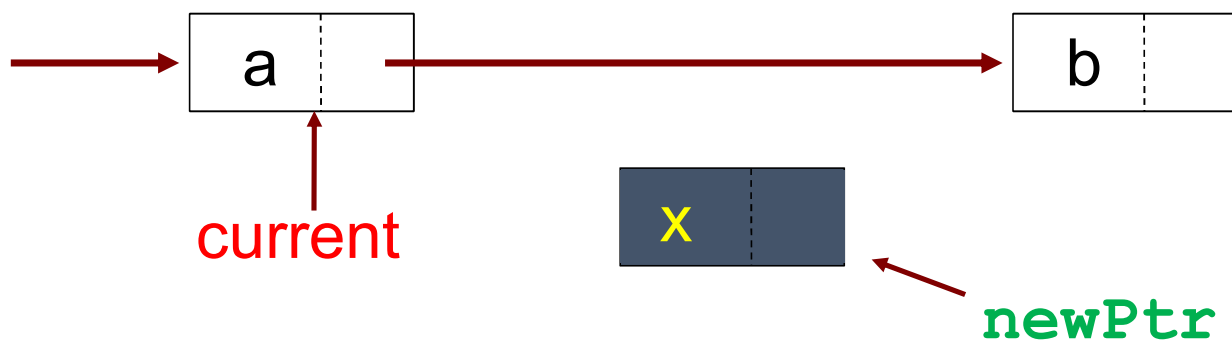
- เพิ่มโหนด X หลังตำแหน่งที่ $current$ ชี้

```
newPtr = malloc(sizeof(LISTNODE))
```



การเพิ่มโหนดในลิงค์ลิสต์

- เพิ่มโหนด X หลังตำแหน่งที่ **current** ชี้
- ```
newPtr = malloc(sizeof(LISTNODE))
newPtr->info = 5
```



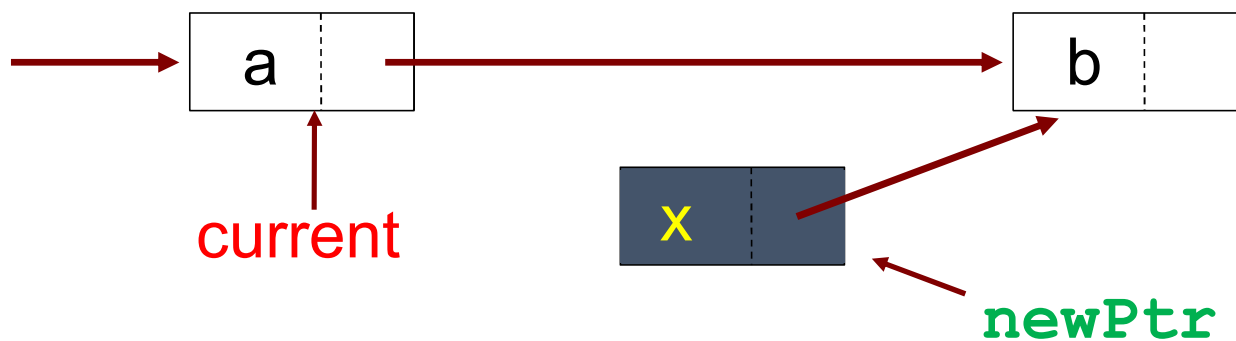
## การเพิ่มโหนดในลิงค์ลิสต์

- เพิ่มโหนด  $X$  หลังตำแหน่งที่  $current$  ชี้

`newPtr = malloc(sizeof(LISTNODE))`

`newPtr->info = 5`

`newPtr.next = current.next;`



## การเพิ่มโหนดในลิงค์ลิสต์

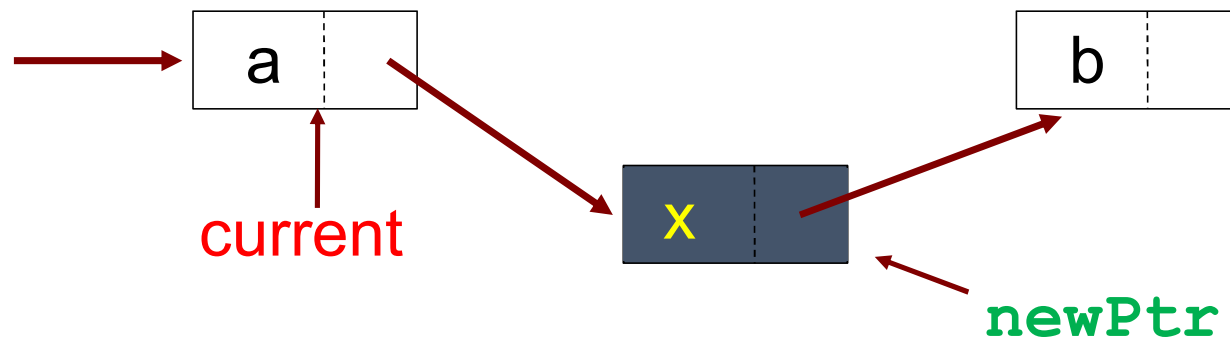
- เพิ่มโหนด X หลังตำแหน่งที่ current ชี้

```
newPtr = malloc(sizeof(LISTNODE))
```

```
newPtr->info = 5
```

```
newPtr->next = current->next;
```

```
current->next = newPtr;
```



## การเพิ่มโหนดในลิงค์ลิสต์ (Append)

- เพิ่มโหนด X ที่ตำแหน่งสุดท้ายของลิงค์ลิสต์

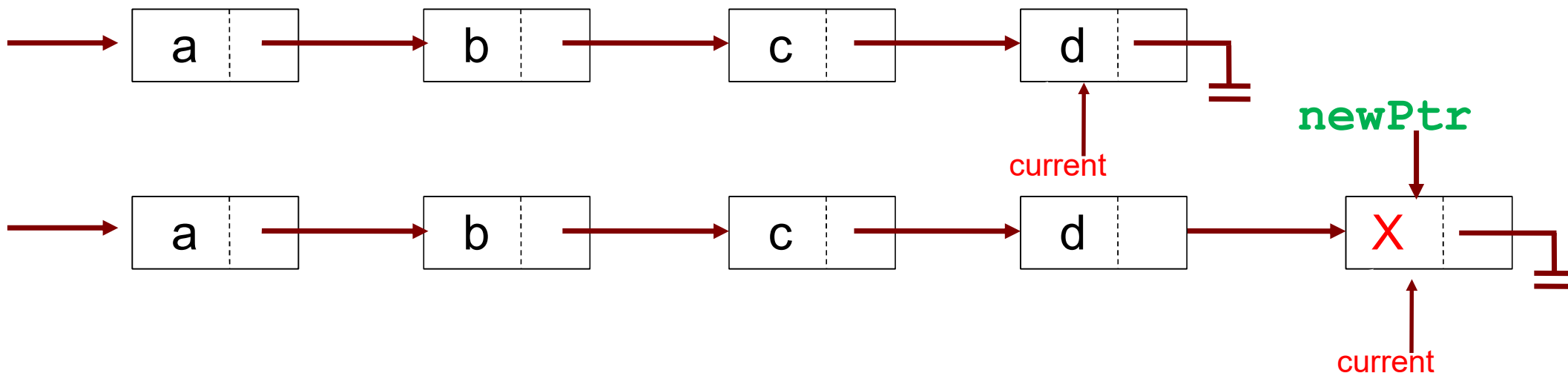
```
newPtr= malloc(sizeof(LISTNODE))
```

```
newPtr->info = X
```

```
current->next = newPtr;
```

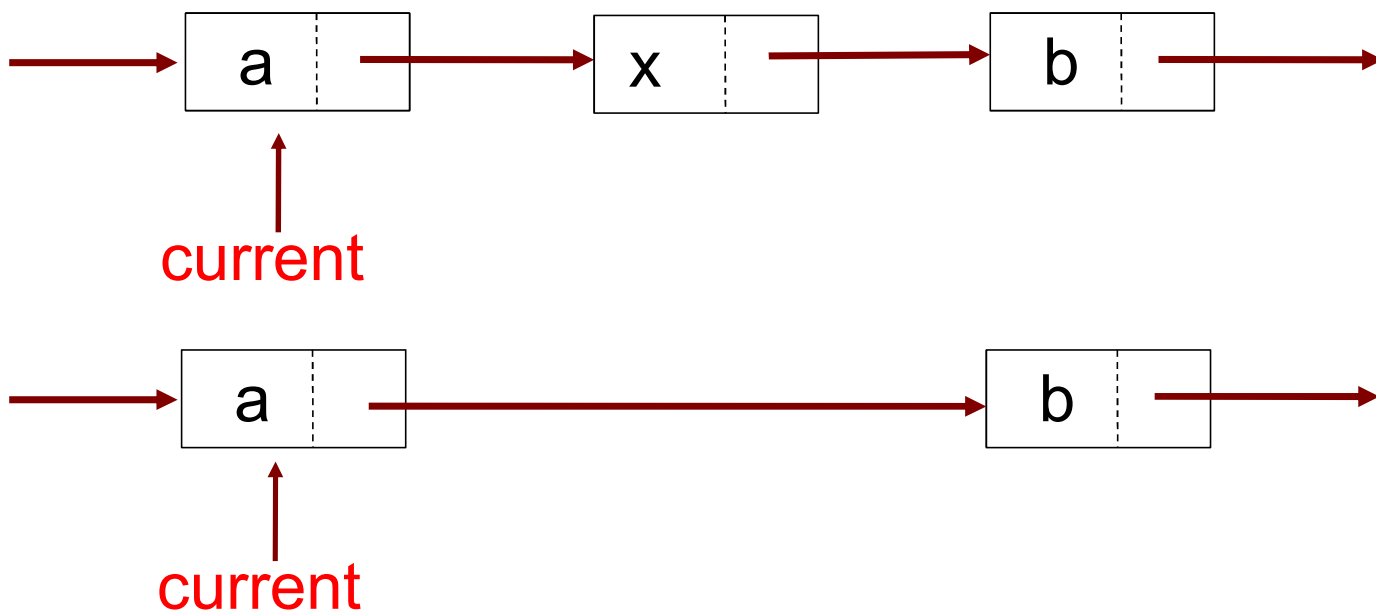
```
current = newPtr;
```

```
current->next = null;
```



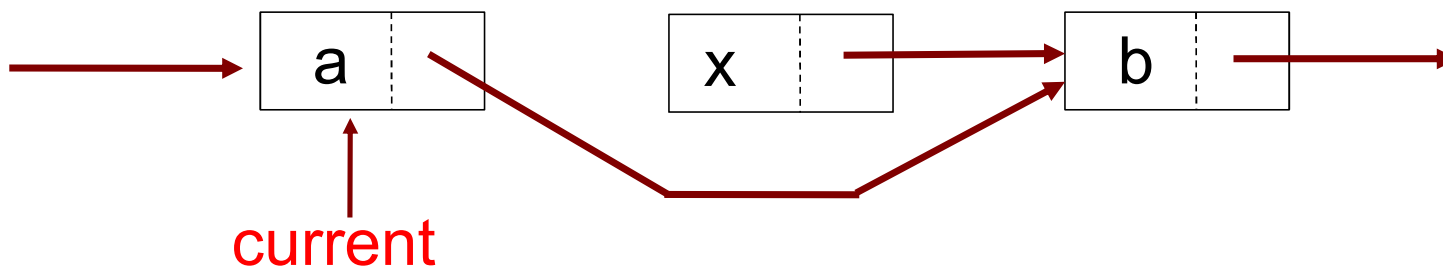
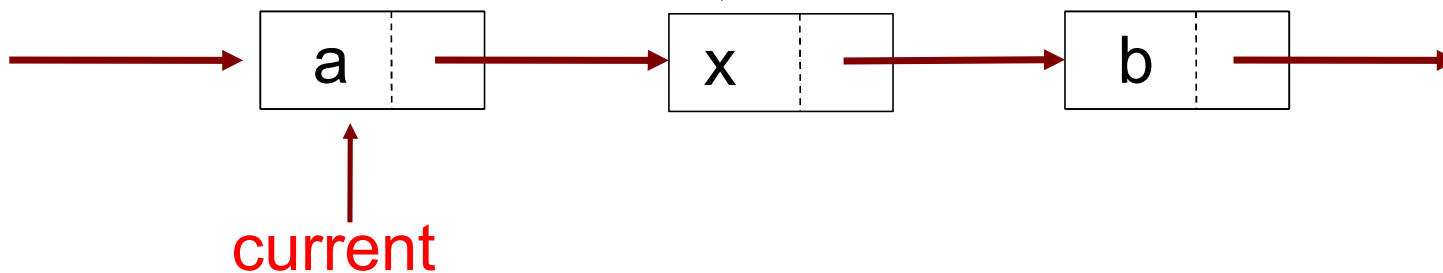
# การลบโหนดในลิงค์ลิสต์

- ลบโหนด X หลังตำแหน่งที่ current ชี้



# การลบโหนดในลิงค์ลิสต์

`current.next = current.next.next;`



## การท่องไปในลิงค์ลิสต์

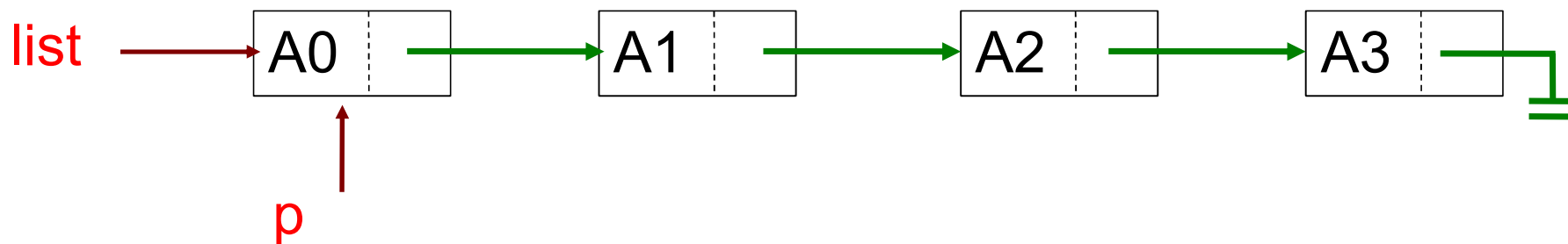
- ถ้าข้อมูลอยู่ในอาร์เรย์:

```
for (index = 0; index < length; index++)
 printf (a[index]);
```

- ถ้าข้อมูลอยู่ในลิงค์ลิสต์

position p

```
for(p=list; p!=null; p=p.next)
 printf (p->info);
```





# Implementing a Linked List

- So what would a Linked List implementation look like?

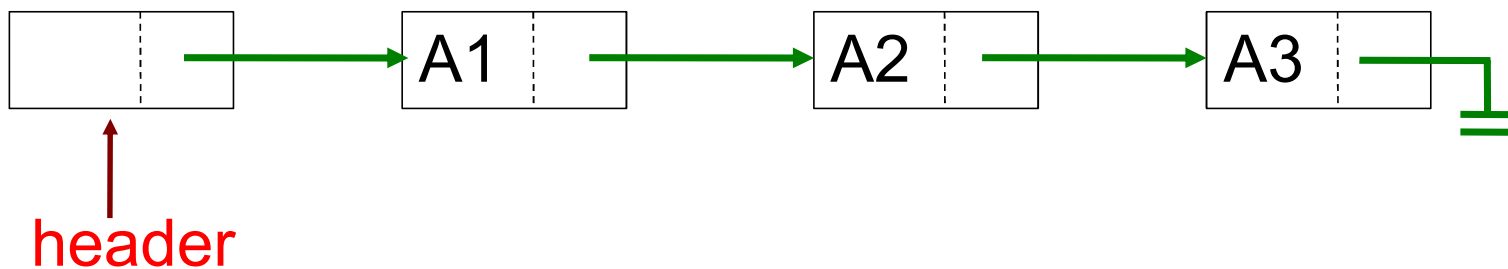
```
typedef struct node *ptrNode;
typedef ptrNode listPtr;
typedef ptrNode position;
```

```
struct node
{
 int info;
 int count;
 position next;
};
```

- จะเกิดอะไรขึ้นถ้า
  - ลบโหนดแรก?
  - เพิ่มข้อมูลหน้าโหนดแรก?

## Header Nodes

- การใส่สมาชิกลงหัวลิสต์หรือการนำสมาชิกออกจากหัวลิสต์เป็นกรณีพิเศษ เราสามารถให้มีหัวลิสต์ปลอม เรียกว่า header หรือ dummy node ได้ ถ้าทำอย่างนี้ทุกๆ โหนดก็จะมีโหนดนำหน้า ทำให้รายละเอียดการโค้ดเหมือนกันหมด
  - เป็นโหนดที่ไม่มีข้อมูลอยู่ภายใน ทำหน้าที่ชี้ไปที่โหนดจริง ๆ ในลิงค์ลิสต์
  - การไปที่สมาชิกตัวแรกของ linked list กำหนดโดยให้ `current` ไปที่ `header.next`;
  - ลิงค์ลิสต์จะว่าง ถ้า `header.next == null`;
- การค้นหาข้อมูลจะข้าม `header`.



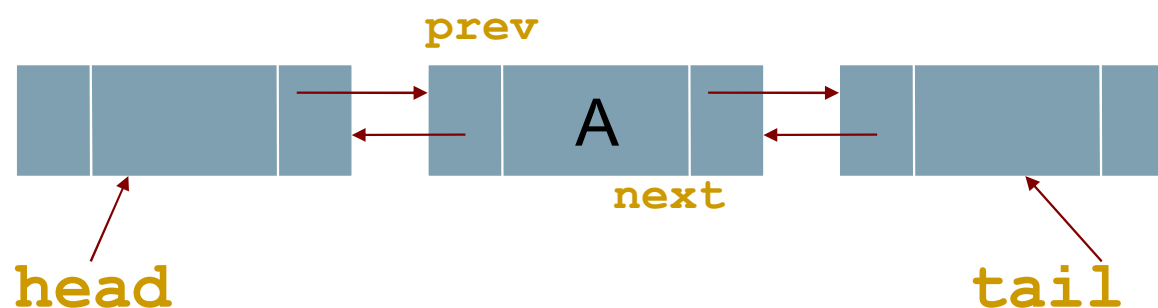


## คุณสมบัติของ Linked List

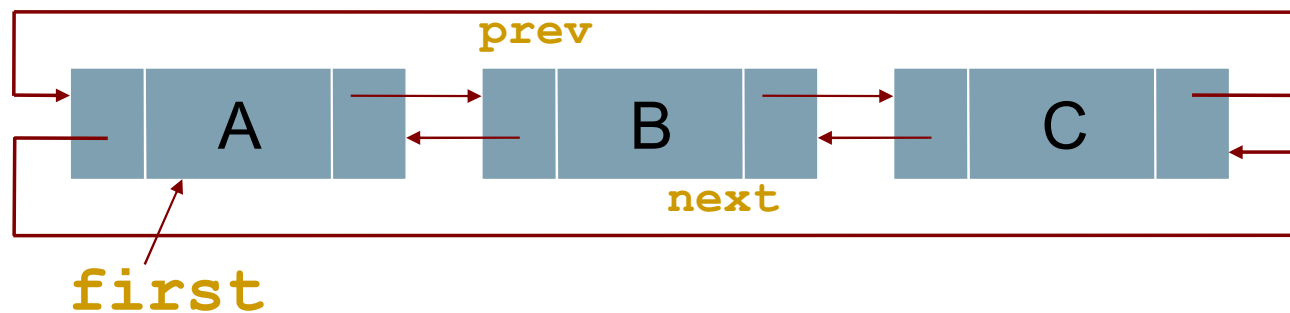
- การวิเคราะห์เวลา Running Time
  - การเพิ่มโหนดต่อจากโหนดก่อนหน้า หรือโหนดสุดท้ายใช้เวลา  $O(1)$
  - การลบโหนดต่อจากโหนดก่อนหน้า หรือโหนดแรกใช้เวลา  $O(1)$
  - การค้นหาข้อมูลใช้เวลา  $O(n)$
  - การดึงข้อมูล ณ ตำแหน่งปัจจุบันใช้เวลา  $O(1)$
- ประโยชน์
  - สามารถเพิ่มโหนดได้เรื่อย ๆ
  - ง่ายต่อการอ่าน ลบ เพิ่ม ข้อมูลของโหนดแรกหรือโหนดสุดท้าย
- ข้อเสีย
  - เสียเวลาในการเรียก `malloc`
  - เสียพื้นที่สำหรับเก็บตัวชี้ที่จะชี้ไปที่ตำแหน่งของโหนดต่อไป

## ลิงค์ลิสต์ประเภทอื่น ๆ

- *Doubly-linked lists*: ทุก ๆ โหนดใน list จะเก็บ pointer ชี้ไปที่โหนดก่อนหน้า และโหนดถัดไป.



- *Circular-linked lists*: โหนดสุดท้ายจะชี้ไปที่โหนดแรกโดยจะใช้ Header ด้วยก็ได้



## Doubly-linked lists: Wrong Insert Next

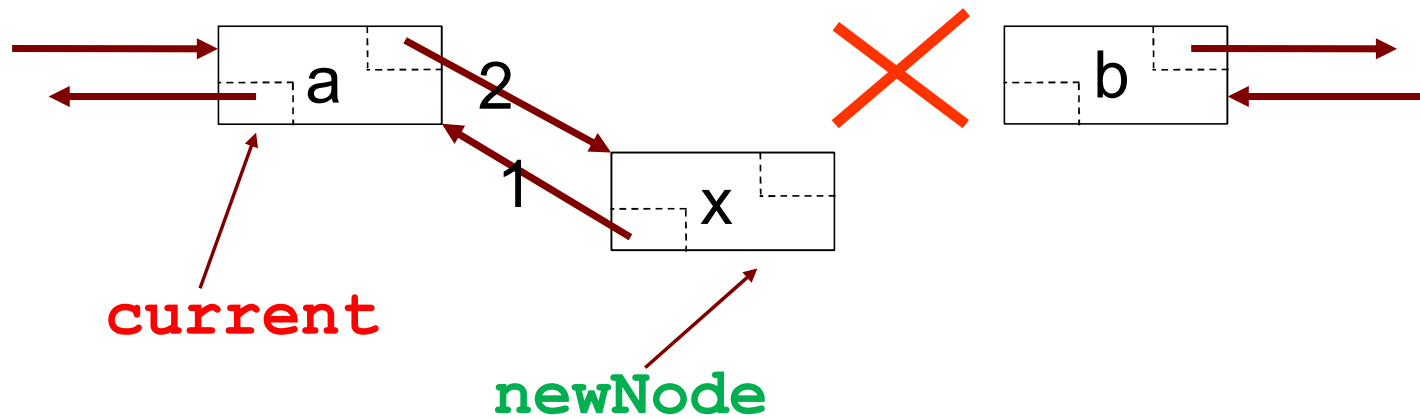
```
newNode = malloc(sizeof(LISTNODE));
```

```
1 newNode.prev = current;
```

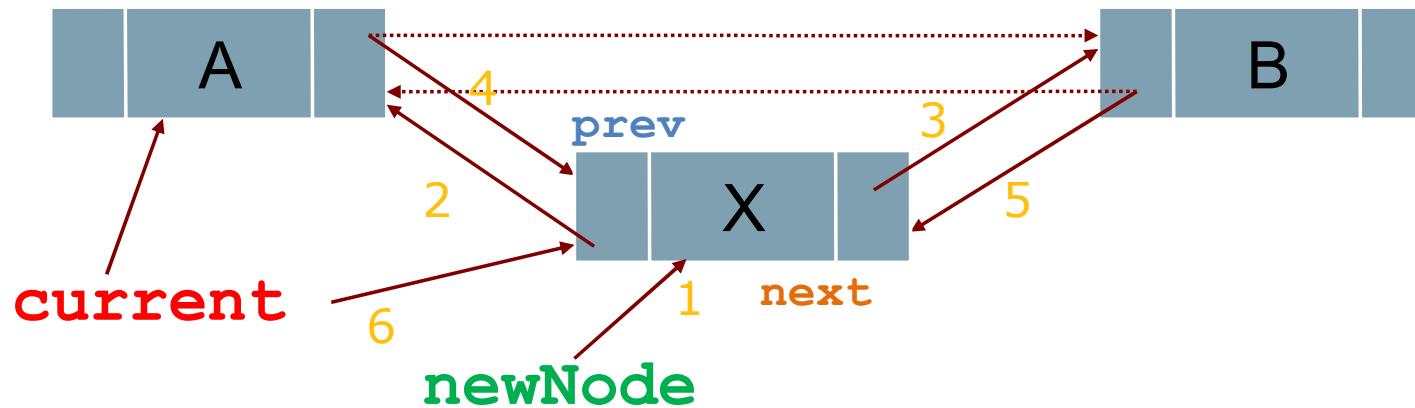
```
2 newNode.prev.next = newNode;
```

...

...



## Doubly-linked lists: insert Next



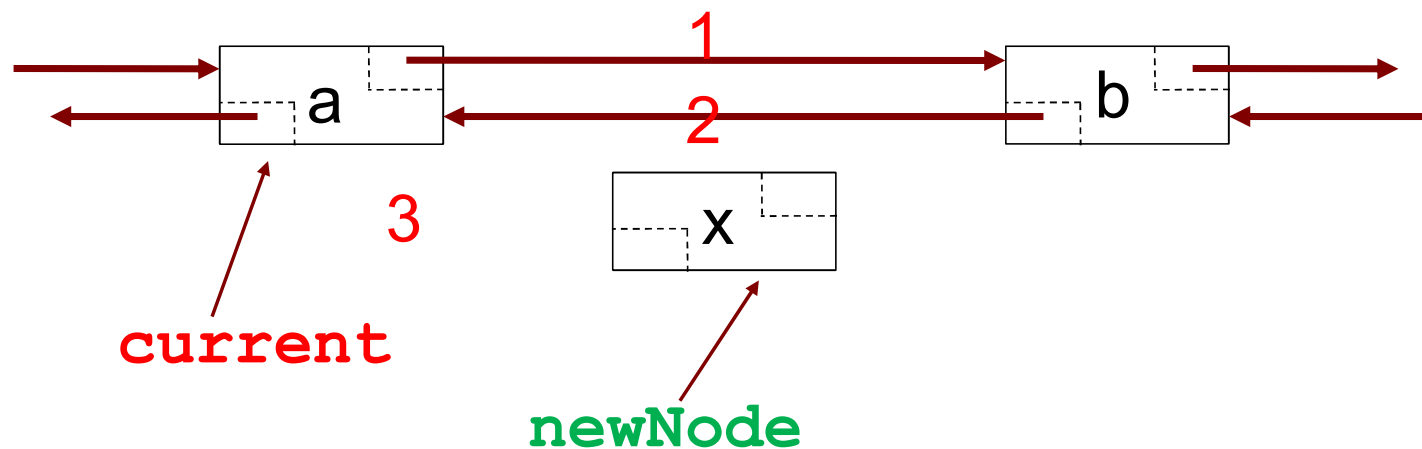
```

1 newNode = malloc(sizeof(LISTNODE)) ;
2 newNode.prev = current;
3 newNode.next = current.next;
4 newNode.prev.next = newNode;
5 newNode.next.prev = newNode;
6 current = newNode;

```

## Doubly-linked lists: Delete Current

- 1 `current.prev.next = current.next;`
- 2 `current.next.prev = current.prev;`
- 3 `current = current.prev;`





## Array versus Linked Lists

- Arrays are suitable for:
  - Inserting/deleting an element at the end.
  - Randomly accessing any element.
  - Searching the list for a particular value.
- Linked lists are suitable for:
  - Inserting an element.
  - Deleting an element.
  - Applications where sequential access is required.
  - In situations where the number of elements cannot be predicted beforehand.

```

#include<bits/stdc++.h>
using namespace std;
int main()
{
 list<int> :: iterator it;
 list<int> l;
 l.push_back(10);
 l.push_back(20);
 l.push_back(30);
 l.push_back(40);
 l.push_back(50);
 l.push_back(60);
 l.push_front(0);
 for(it = l.begin() ; it != l.end() ; it++)
 {
 cout<< *it <<"\n";
 }
 cout<<endl;

 it = find(l.begin(), l.end(), 40); //insert before 40
 l.insert(it, 44);
 for(it = l.begin() ; it != l.end() ; it++)
 {
 cout<< *it <<"\n";
 }
 cout<<endl;

 l.remove(40); //remove 40
 for(it = l.begin() ; it != l.end() ; it++)
 {
 cout<< *it <<"\n";
 }
}

```

# Pointer

```
#include<bits/stdc++.h>
using namespace std;
int strlen (char *s)
{
 char *p = s;
 while (*p != '\0') { p++; }
 return p-s;
}
void strcpy (char *s, char *t)
{
 int i=0;
 while((s[i] = t[i]) != '\0') { i++; } //while ((*s = *t) != '\0'){s++;t++;}
}
void sum_value(int a, int b) //pass by value
{
 a = a + b;
 cout<< a <<endl;
}
void sum_ref(int *a, int *b) //pass by reference
{
 *a = *a + *b;
 cout<< *a <<endl;
}

int main()
{
 int i = 10;
 int *p1 = &i;

 int j = 20;
 int *p2 = &j;

 cout<<i<<" "<<&i<<endl;
 cout<<*p1<<" "<<p1<<" "<<&p1<<endl;

 cout<<j<<" "<<&j<<endl;
 cout<<*p2<<" "<<p2<<" "<<&p2<<endl;

 i = 30;
 cout<<i<<" "<<*p1<<endl;

 *p1 = 40;
 cout<<i<<" "<<*p1<<endl;
```

```

int a[5] = {1,2,3,4,5};
int *pa;
pa = &a[0]; //connect
pa = a; //connect
cout<< *(pa+1) <<" "<< pa[1] <<endl;

```

```

char *pm = "Hello, world";
char am[] = "Hello, world";

```

```

cout<<strlen (pm) <<" " << strlen (am)<<endl;

```

```

typedef struct today
{
 int num;
 char message[100];
};
today t1, *t2;

```

```

t1.num = 20;
strcpy(t1.message, "C Programming");
t2 = &t1;

```

```

cout<<t1.num<<" "<<t2->num<<" "<<t1.message <<" "<<t2->message <<endl;

```

```

int s1 = 10, s2 = 20;
sum_value(s1, s2);
cout<<s1<<endl;
sum_ref(&s1, &s2);
cout<<s1<<endl;

```

```

char str1[20] = "C programming";
char str2[20];
strcpy(str2, str1);

```

```

cout<<strcmp(str1, str2)<<endl;

```

```

}

```